

TTDS 2025 Past Paper Deep Solutions

Original-question guide + worked solutions

Generated for TTDS revision

2026-05-20

Contents

使用方式	1
Question 1: Mixed Short/Calculation Questions	2
Question 2: Non-deterministic Positional Inverted Index for ASR	5
Question 3: Expected Mutual Information	8
Question 4: Short Answers	10
Question 5: PageRank without Teleportation	12
Question 6: Coding Assistant RAG Design	14
Coverage Checklist	15

使用方式

这份解析对应 2025389_INFR11145_INFR11229.pdf。2025 卷更强调 applied IR: LLM vs search、RAG、ASR/noisy indexing、PageRank without teleportation、coding assistant design。中文讲解为主，关键 TTDS terms 保留英文。

Question 1: Mixed Short/Calculation Questions

Original question screenshot

Please provide answers to **ALL** of the following questions. It is more important to have a clear answer than a longer answer. When calculations are needed, please **show steps** and give answer to **three** decimal digits.

1. (a) For each of the following preprocessing steps, explain the step, and list an advantage and a drawback with an example of each situation: [3 marks]
 - i. Stopping
 - ii. Removing URLs (e.g. URL in a tweet text)
- (b) “With the advent of LLMs, web search engines are no longer needed”. Respond to the previous statement by agreeing or disagreeing while discussing your justifications [2 marks]
- (c) In the TFIDF formula, $\log_{10}$ is usually used in calculating the tf and df components. If someone decided to replace it with $\log_2$:
 - i. How should this impact the values of TF and IDF? [2 marks]
 - ii. How do you think this can affect the ranking of documents? [2 marks]
- (d) Regarding Heap’s law, which is expected to have smaller b value, a closed-domain collection or a general-domain collection? Why? [2 marks]
- (e) System X retrieved a set of documents to a given query, where the relevance of top 5 ranked documents are: {2, 1, 3, 0, 3} in order.
 - i. Assume the collection have many highly relevant documents with relevance 3, calculate $nDCG@5$. [2 marks]
 - ii. Assuming only documents with relevance 3 to be considered relevant, calculate MRR. [1 mark]
- (f) Spiders crawl webpages to build an index for search engines. However, some websites get hacked, replacing their original content with irrelevant content.
 - i. Why this is a problem? [1 mark]
 - ii. What key mechanism allows spiders to detect and mitigate this issue? [2 marks]
- (g) You have a large-scale Retrieval-Augmented Generation (RAG) system with query and document encoders that generate embeddings for documents. You are tasked with improving the system by updating the encoders. Why it is NOT practical to update the document encoder? [3 marks]

原题要点

考 stopping、removing URLs、LLM vs search engines、log base in TFIDF、Heap’s law、 $nDCG$ /MRR、crawler freshness/security、RAG encoder update cost.

(a) Preprocessing

Stopping / stopword removal

Definition: remove very frequent function words such as the, is, of.

Advantage: reduces index size and noise for bag-of-words retrieval.

Drawback: can remove meaning in phrase queries or tasks where function words matter, e.g. “to be or not to be”, “The Who”, or negation-like constructions.

Removing URLs in tweets

Definition: delete URL strings from tweet text before indexing/classification.

Advantage: reduces sparse, unique tokens and prevents the model from overfitting to arbitrary links.

Drawback: URLs may contain important evidence, source credibility, domain names, campaign links, or misinformation signals.

(b) “LLMs make search engines unnecessary”

Strong answer: disagree, but say LLMs change search interfaces.

Search engines remain needed because they provide:

- Freshness: updated web index
- Attribution: URLs/snippets for verification
- Navigational search: users often want a specific website
- Transactional search: buying, booking, downloading
- Coverage: web-scale corpus not stored fully in model parameters

LLMs help by summarising, comparing, explaining, and supporting conversational search. Best architecture is often RAG: retrieve evidence, then generate grounded answer.

(c) Replacing log10 with log2 in TFIDF

For any $x > 0$:

$$\log_2 x = \frac{\log_{10} x}{\log_{10} 2}$$

So replacing log10 by log2 multiplies TF/IDF log values by a positive constant. If every score is multiplied by the same positive constant, rankings usually do not change. If both tf and idf log components are changed, scores may be scaled by constant factors, but monotonic ordering is preserved under simple linear TFIDF. Small numerical differences do not change rank unless there are ties or mixed features with other unscaled components.

(d) Heap’ s law: smaller b

Heap’ s law:

$$V = kN^b$$

Closed-domain collection should have smaller b because vocabulary growth slows sooner: documents reuse a specialised vocabulary. General-domain collection keeps introducing new topics, names, and terms, so vocabulary grows faster.

(e) nDCG@5 and MRR

Top 5 graded relevance:

$$[2, 1, 3, 0, 3]$$

Assume many ideal relevance-3 documents exist, so ideal top 5:

$$[3, 3, 3, 3, 3]$$

Using:

$$DCG@5 = \sum_{i=1}^5 \frac{gain_i}{\log_2(i+1)}$$

System:

$$DCG@5 = 2 + \frac{1}{\log_2 3} + \frac{3}{\log_2 4} + 0 + \frac{3}{\log_2 6} = 5.291$$

Ideal, because the question says the collection has many highly relevant documents with relevance 3:

$$IDCG@5 = 3 + \frac{3}{\log_2 3} + \frac{3}{\log_2 4} + \frac{3}{\log_2 5} + \frac{3}{\log_2 6} = 8.845$$

$$nDCG@5 = 5.291/8.845 = 0.598$$

For MRR, only relevance 3 counts as relevant. First relevance-3 document is at rank 3:

$$MRR = \frac{1}{3} = 0.333$$

(f) Hacked websites and crawler mitigation

Problem: hacked content pollutes index, hurts ranking quality, may serve spam/malware, and breaks trust because the page no longer matches its prior content.

Key mechanism: freshness/re-crawling plus change detection. Spiders revisit pages, compare content/signatures, detect abnormal changes, and update or remove stale/spam content. Link analysis and spam detection may also help.

(g) Why not update document encoder in large RAG

In dense retrieval/RAG, document embeddings are precomputed using the document encoder and stored in a vector database. Updating the document encoder means all document embeddings become stale and the whole corpus must be re-embedded and re-indexed, which is expensive at scale. Updating query encoder or reranker is often more practical.

Question 2: Non-deterministic Positional Inverted Index for ASR

Original question screenshot

2. The Parliament in a given country has decided to record all their sessions and make them searchable. They have hired you as an IR expert to build the engine for them. You have noticed that the automatic speech recognition (ASR) systems available in the market for their language have poor performance (50% word error rate). However, you have also noticed that if you make the ASR system produce the most likely words for each audio segment, there is a high chance (90%) of finding the correct word among them. Thus, you have decided to design a new *non-deterministic* inverted index that enables indexing the *three* most likely words for each audio segment along with their probabilities. Below is an example of the ASR output that corresponds to the sentence “**creating jobs is our top priority**”, where each term in the document is represented as a vector of the most likely words along with their probabilities.

w ₁		w ₂		w ₃		w ₄		w ₅		w ₆	
treating	0.4	jobs	0.35	is	0.7	hour	0.55	top	0.45	security	0.3
mitigating	0.3	hobs	0.3	oz	0.2	our	0.3	good	0.4	minority	0.3
deleting	0.15	dobs	0.25	ezzy	0.05	awa	0.1	job	0.1	priority	0.25
creating	0.15	ops	0.1	it	0.05	out	0.05	drop	0.05	seniority	0.05

- (a) Please propose a design of a positional inverted index that can index this document representation. Illustrate your answer by giving examples of what the posting list would look like. [4 marks]
- (b) Would proximity search using your proposed approach work the same as it does with a standard index (with deterministic terms)? [1 mark]
- (c) Explain how *tf*, *df*, and *cf* can be calculated in this situation, and how each is different from the normal situation (when terms are deterministic). [2 marks]
- (d) Suppose the above phrase (w₁ to w₆) is part of a document with no other occurrences of these terms in the document. What is the *tf* score of the query “Job creation” after applying standard preprocessing and ignoring IDF? [1 mark]
- (e) How do you think the performance of this system compares to a standard system –where only most probable recognised term is indexed– in terms of precision and recall? [2 marks]

原题要点

ASR has poor 1-best performance but top-4 candidates per word position often contain the correct word. Need design a positional inverted index that stores candidate terms with probabilities.

Example positions:

Position	Candidate terms with probabilities
w1	treating .4, mitigating .3, deleting .15, creating .15
w2	jobs .35, hobs .3, dobs .25, ops .1
w3	is .7, oz .2, ezzy .05, it .05
w4	hour .55, our .3, awa .1, out .05
w5	top .45, good .4, job .1, drop .05
w6	security .3, minority .3, priority .25, seniority .05

(a) Index design

Use a probabilistic positional inverted index. For each term, posting stores:

$$(docID, position, probability)$$

Example postings for this document d1:

- creating → (d1, w1, 0.15)
- jobs → (d1, w2, 0.35)
- job → (d1, w5, 0.10)
- our → (d1, w4, 0.30)
- top → (d1, w5, 0.45)
- priority → (d1, w6, 0.25)

For a query, scoring can sum or multiply probabilities depending on retrieval model. A phrase/proximity match can check positions while incorporating uncertainty.

(b) Does proximity search work the same?

Not exactly. Standard positional search checks deterministic positions of exact terms. Here each position contains multiple possible terms with probabilities, so phrase/proximity matching becomes probabilistic. A phrase may match with low or high confidence depending on candidate probabilities.

(c) tf, df, cf in this setting

There are two possible definitions; a good answer should state the design.

- $tf(term, document)$: sum probabilities of term over all positions in the document.
- $df(term)$: number of documents where term appears with non-zero probability, or expected df using a threshold/probability sum.
- $cf(term)$: sum probabilities over all positions in the collection.

Difference from normal case: counts are fractional/expected rather than integer deterministic counts.

(d) tf score for query “Job creation”

After preprocessing:

- Job likely normalises/stems to job/jobs
- creation likely stems to creat, matching creating

Ignoring IDF and using expected tf:

- For job: jobs at w2 has 0.35 and job at w5 has 0.10, so expected $tf = 0.45$ if stemming conflates job/jobs.
- For creation/creating: creating at w1 has 0.15.

Total score:

$$tf = 0.45 + 0.15 = 0.60$$

If no stemming/singular-plural conflation is used, only exact job contributes 0.10 and exact creation contributes 0, so answer should explicitly state preprocessing assumptions. The question says after standard preprocessing, so stemming-normalisation answer 0.60 is stronger.

(e) Precision/recall vs standard 1-best system

Compared with indexing only the most probable ASR word, this system should improve recall because correct words that are not top-1 can still be retrieved. Precision may decrease because extra candidate terms add false matches. Probability-aware ranking can recover some precision by downweighting uncertain candidates.

Question 3: Expected Mutual Information

Original question screenshot

3. You have listened to music. A lot of music. You would like to know how often certain words appear in certain types of songs. For simplicity, you have assigned each song to exactly one genre, such as *pop* or *rock*. You have identified a number of words that are of interest to you, such as *moon* and *tears*, and counted how many songs from each genre they appear in:

	Pop	Rock	Hip-hop	Country	Jazz	Electronic
Total no. of songs	403	1243	756	1202	341	1272
love	85	602	43	769	55	132
night	92	48	337	25	71	115
dance	123	52	24	176	68	422
heart	78	113	44	84	153	202
dream	182	23	15	67	95	182
moon	98	93	12	15	102	152
tears	57	83	105	5	79	38

Calculate the expected mutual information $I(U; C)$ for the word *heart* and the genre *hip hop* (show steps).

$$\begin{aligned} I(U; C) &= \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)} \\ &= \frac{N_{11}}{N} \log_2 \frac{NN_{11}}{N_{1.}N_{.1}} + \frac{N_{01}}{N} \log_2 \frac{NN_{01}}{N_{0.}N_{.1}} + \frac{N_{10}}{N} \log_2 \frac{NN_{10}}{N_{1.}N_{.0}} + \frac{N_{00}}{N} \log_2 \frac{NN_{00}}{N_{0.}N_{.0}} \end{aligned}$$

[5 marks]

4. Provide a short answer to each of the following questions:

- (a) The word “latent” means “hidden” or, in the context of Latent Dirichlet Allocation “not (directly) observable”. What does it refer to? Describe briefly what is not observable (latent), and what is observable in LDA.
- (b) Describe briefly a situation where it makes sense to use a sentiment dictionary instead of a machine learning model.
- (c) Where do the topic labels in topic modelling (using LDA) come from?
- (d) In classification, what does “multi-class” mean, and what does “multi-label” mean?
- (e) Describe the difference between a test set and a validation test. When do we use them and why do we need both?

[5 marks]

原题要点

Compute MI for word heart and genre Hip-hop.

Totals:

$$N = 403 + 1243 + 756 + 1202 + 341 + 1272 = 5217$$

For heart:

- $N_{11} = 44$
- $N_{1.} = 78 + 113 + 44 + 84 + 153 + 202 = 674$
- $N_{.1} = 756$
- $N_{10} = 630$
- $N_{01} = 712$
- $N_{00} = 3831$

Calculation

Cell	Count	Contribution
N_{11}	44	-0.009703
N_{01}	712	0.015431
N_{10}	630	0.015512
N_{00}	3831	-0.014739

$$I(\text{heart}; \text{HipHop}) = 0.006502 \approx 0.007$$

Interpretation

The value is low. heart is not strongly associated with Hip-hop; it appears across many genres. MI rewards class-specific association, not raw frequency alone.

Question 4: Short Answers

Original question screenshot

3. You have listened to music. A lot of music. You would like to know how often certain words appear in certain types of songs. For simplicity, you have assigned each song to exactly one genre, such as *pop* or *rock*. You have identified a number of words that are of interest to you, such as *moon* and *tears*, and counted how many songs from each genre they appear in:

	Pop	Rock	Hip-hop	Country	Jazz	Electronic
Total no. of songs	403	1243	756	1202	341	1272
love	85	602	43	769	55	132
night	92	48	337	25	71	115
dance	123	52	24	176	68	422
heart	78	113	44	84	153	202
dream	182	23	15	67	95	182
moon	98	93	12	15	102	152
tears	57	83	105	5	79	38

Calculate the expected mutual information $I(U; C)$ for the word *heart* and the genre *hip hop* (show steps).

$$\begin{aligned} I(U; C) &= \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)} \\ &= \frac{N_{11}}{N} \log_2 \frac{NN_{11}}{N_{1.}N_{.1}} + \frac{N_{01}}{N} \log_2 \frac{NN_{01}}{N_{0.}N_{.1}} + \frac{N_{10}}{N} \log_2 \frac{NN_{10}}{N_{1.}N_{.0}} + \frac{N_{00}}{N} \log_2 \frac{NN_{00}}{N_{0.}N_{.0}} \end{aligned}$$

[5 marks]

4. Provide a short answer to each of the following questions:

- The word “latent” means “hidden” or, in the context of Latent Dirichlet Allocation “not (directly) observable”. What does it refer to? Describe briefly what is not observable (latent), and what is observable in LDA.
- Describe briefly a situation where it makes sense to use a sentiment dictionary instead of a machine learning model.
- Where do the topic labels in topic modelling (using LDA) come from?
- In classification, what does “multi-class” mean, and what does “multi-label” mean?
- Describe the difference between a test set and a validation test. When do we use them and why do we need both?

[5 marks]

(a) What is latent in LDA?

In LDA, latent refers to hidden variables not directly observed:

- topic assignment for each word token
- topic distribution for each document
- word distribution for each topic

Observable: the words in documents. We observe tokens, not the topics that generated them.

(b) When use sentiment dictionary instead of ML?

Use a sentiment dictionary when labelled data is unavailable, transparency matters, domain is simple, or we need quick interpretable aggregate analysis. Example: counting positive/negative words in survey comments. Draw-back: dictionaries miss context, sarcasm, negation, and domain-specific meanings.

(c) Where do topic labels come from?

LDA outputs topics as word distributions, not human-readable labels. Topic labels are assigned by humans after inspecting top words/documents, or generated by an external labelling process. They are not directly learned as semantic labels by vanilla LDA.

(d) Multi-class vs multi-label

Multi-class: each instance belongs to exactly one class among more than two classes, e.g. topic = sports/business/politics.

Multi-label: each instance may belong to multiple classes simultaneously, e.g. a document can be both politics and economy.

(e) Test set vs validation test/dev set

Validation/development set is used during model development for hyperparameter tuning, feature selection, threshold choice, and model selection.

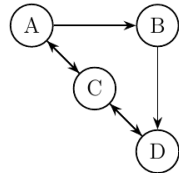
Test set is held out for final unbiased evaluation after all choices are fixed.

Need both because tuning on test data contaminates the final estimate.

Question 5: PageRank without Teleportation

Original question screenshot

5. Consider the directed graph below. Apply the PageRank algorithm without teleportation. That is, the random surfer never jumps to a random node in an iteration. Answer the questions below. Show your work.



- (a) What are the rankings of nodes (descending) after the 1st iteration? [2 marks]
- (b) What are the rankings of nodes (descending) after the 2nd iteration? [2 marks]
- (c) Explain why we should or do not need to employ teleportation? [1 mark]
6. You are designing a coding assistant named ElmasAI, which is a Retrieval-Augmented Generation (RAG) system that integrates with a user's development environment (e.g., Visual Studio). It has access to local files on your computer. It employs OpenAI's GPT model through API calls.

ElmasAI provides real-time feedback on code mistakes and can generate code upon request. It should perform well in projects with multiple code files deployed locally.

Answer the following questions:

- (a) Describe the potential data and data sources to train and fine-tune ElmasAI [1 mark]
- (b) Apart from the data that is used to train and fine-tune ElmasAI, what are the documents ElmasAI should use to create the vector database upon integration? [2 marks]
- (c) Your users have security and privacy concerns because ElmasAI requires access to the internet to work! How could you modify ElmasAI to address this concern for good? What is your proposed solution's drawback? [2 marks]

原题要点

Graph from the figure:

- A → B
- A → C
- B → D
- C → A
- C → D

- D → C

No teleportation, so update is:

$$PR_{i+1}(x) = \sum_{y \rightarrow x} \frac{PR_i(y)}{L_{out}(y)}$$

Initialise uniformly:

$$PR_0(A) = PR_0(B) = PR_0(C) = PR_0(D) = 0.25$$

(a) After 1st iteration

- $PR_1(A) = PR_0(C)/2 = 0.125$
- $PR_1(B) = PR_0(A)/2 = 0.125$
- $PR_1(C) = PR_0(A)/2 + PR_0(D) = 0.125 + 0.25 = 0.375$
- $PR_1(D) = PR_0(B) + PR_0(C)/2 = 0.25 + 0.125 = 0.375$

Ranking:

$$C = D > A = B$$

(b) After 2nd iteration

- $PR_2(A) = PR_1(C)/2 = 0.1875$
- $PR_2(B) = PR_1(A)/2 = 0.0625$
- $PR_2(C) = PR_1(A)/2 + PR_1(D) = 0.0625 + 0.375 = 0.4375$
- $PR_2(D) = PR_1(B) + PR_1(C)/2 = 0.125 + 0.1875 = 0.3125$

Ranking:

$$C > D > A > B$$

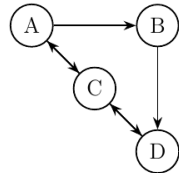
(c) Why teleportation is needed

Teleportation prevents rank sinks, dead ends, and disconnected components from trapping the random surfer. Without teleportation, PageRank may fail to reflect global importance or may concentrate inside cycles. In this particular graph there is no dangling node, but teleportation is still generally used for robustness and convergence to a unique stationary distribution.

Question 6: Coding Assistant RAG Design

Original question screenshot

5. Consider the directed graph below. Apply the PageRank algorithm without teleportation. That is, the random surfer never jumps to a random node in an iteration. Answer the questions below. Show your work.



- (a) What are the rankings of nodes (descending) after the 1st iteration? [2 marks]
 - (b) What are the rankings of nodes (descending) after the 2nd iteration? [2 marks]
 - (c) Explain why we should or do not need to employ teleportation? [1 mark]
6. You are designing a coding assistant named ElmasAI, which is a Retrieval-Augmented Generation (RAG) system that integrates with a user's development environment (e.g., Visual Studio). It has access to local files on your computer. It employs OpenAI's GPT model through API calls.

ElmasAI provides real-time feedback on code mistakes and can generate code upon request. It should perform well in projects with multiple code files deployed locally.

Answer the following questions:

- (a) Describe the potential data and data sources to train and fine-tune ElmasAI [1 mark]
- (b) Apart from the data that is used to train and fine-tune ElmasAI, what are the documents ElmasAI should use to create the vector database upon integration? [2 marks]
- (c) Your users have security and privacy concerns because ElmasAI requires access to the internet to work! How could you modify ElmasAI to address this concern for good? What is your proposed solution's drawback? [2 marks]

原题要点

ElmasAI is a RAG coding assistant integrated with local IDE/files and using GPT API. Questions ask about training data, vector database documents, and privacy-preserving redesign.

(a) Data and sources to train/fine-tune

Potential data:

- open-source code repositories with permissive licences
- code-comment/docstring pairs
- bug-fix commits and pull requests
- Stack Overflow / Q&A style programming data, if licence permits
- instruction-following coding datasets
- IDE interaction logs or synthetic coding tasks

Need to consider licence, privacy, security, and data quality.

(b) Documents for vector database upon integration

The vector DB should index project-specific non-parametric memory:

- local source files in the user project
- README, docs, API references, configuration files
- dependency manifests, schemas, tests
- relevant local coding standards or issue descriptions

It should not only index generic training data; it must retrieve current project context.

(c) Address internet/privacy concern

Solution: run the model locally or inside the organisation's private infrastructure, and keep embeddings/vector DB local. Use a local LLM or self-hosted model instead of sending code to OpenAI API.

Drawback: local/self-hosted models may be weaker, slower, more expensive to maintain, and require hardware. Updating models and security patches becomes the user/organisation's responsibility.

考试模板

For RAG design questions, separate:

1. parametric memory: model training/fine-tuning data
2. non-parametric memory: vector DB documents retrieved at runtime
3. privacy/security: where inference and embedding happen
4. tradeoff: performance/convenience vs local control

Coverage Checklist

Question	Covered TTDS topics
Q1	Preprocessing, LLM vs search, TFIDF log bases, Heap's law, nDCG, MRR, crawling freshness/security, RAG encoders
Q2	Positional inverted index, ASR uncertainty, probabilistic postings, tf/df/cf, precision-recall tradeoff
Q3	Expected mutual information, feature selection, contingency table
Q4	LDA latent variables, sentiment lexicons, topic labels, multi-class vs multi-label, validation vs test
Q5	PageRank without teleportation, graph iteration, random surfer robustness
Q6	RAG architecture, vector database, local project documents, privacy-preserving deployment